

Главная загрузочная запись

А **главная загрузочная запись (MBR)** — это тип [загрузочного сектора](#) в первом блоке [разделенных на разделы](#) компьютерных [устройств хранения данных](#), таких как [жесткие диски](#) или [съемные диски](#), предназначенных для использования в [IBM PC-совместимых](#) системах и других устройствах. Концепция главной загрузочной записи была публично представлена в 1983 году с выходом [PC DOS 2.0](#).

Главная загрузочная запись содержит информацию о том, как секторы диска (также называемые «блоками») разделены на разделы, каждый из которых условно содержит файловую систему. Главная загрузочная запись также содержит исполняемый код, который выполняет функцию загрузчика установленной операционной системы — обычно путем передачи управления второму этапу загрузчика или в сочетании с загрузочной записью тома (VBR) каждого раздела. Этот код MBR обычно называют загрузчиком.

Организация таблицы разделов в MBR ограничивает максимальный объем адресуемого дискового пространства разделенного диска до 2 [TiB](#) ($2^{32} \times 512$ байтов).^[1] Подходы, позволяющие немного увеличить этот предел за счет использования 32-битной арифметики или секторов размером 4096 байт, официально не поддерживаются, так как они нарушают совместимость с существующими загрузчиками, большинством операционных систем, совместимых с MBR, и соответствующими системными инструментами, а также могут привести к серьезному повреждению данных при использовании за пределами строго контролируемых системных сред. Таким образом, схема разбиения на разделы на основе MBR была вытеснена схемой [таблицы разделов GUID](#) (GPT) почти во всех новых компьютерах. GPT может сосуществовать с MBR, обеспечивая ограниченную обратную совместимость со старыми системами.

MBR не используются на носителях без разбиения на разделы, таких как [дискеты](#), [супердискеты](#) или другие устройства хранения данных, настроенные соответствующим образом, а также не обязательно присутствуют на дисках, используемых на платформах, отличных от ПК.

Обзор

Поддержка разделов на носителях и, соответственно, главной загрузочной записи (MBR) была представлена в IBM [PC DOS 2.0](#) в марте 1983 года для поддержки 10 МБ [жесткого диска](#) нового на тот момент [IBM Personal Computer XT](#), который все еще использовал файловую систему [FAT12](#). Первоначальная версия MBR была написана Дэвидом

Литтоном из IBM в июне 1982 года. Таблица разделов поддерживала до четырех *основных разделов*. Ситуация не изменилась, когда в DOS 3.0 появилась новая файловая система **FAT16**. Поддержка *расширенного раздела*, специального типа основного раздела, используемого в качестве контейнера для других разделов, была добавлена в DOS 3.2, а вложенные *логические диски* внутри расширенного раздела появились в DOS 3.30. Поскольку MS-DOS, PC DOS, OS/2 и Windows никогда не загружались с них, формат MBR и загрузочный код практически не менялись по функциональности (за исключением некоторых сторонних реализаций) на протяжении всего периода существования DOS и OS/2 вплоть до 1996 года.

В 1996 году поддержка *адресации логических блоков* (LBA) была введена в Windows 95B и MS-DOS 7.10 (не путать с IBM PC-DOS 7.1) для поддержки дисков размером более 8 ГБ. *Также были введены временные метки на диске.*^[2] Это также отразило идею о том, что MBR должен быть независимым от операционной и файловой систем. Однако это правило проектирования было частично нарушено в более поздних версиях MBR от Microsoft, которые обеспечивают доступ **CHS** для разделов **FAT16B** и **FAT32 0x06/0x0B**, в то время как LBA используется для **0x0E/0x0C**.

Несмотря на то, что некоторые внутренние особенности формата MBR не были должным образом задокументированы (что иногда приводило к проблемам с совместимостью), он стал отраслевым стандартом де-факто благодаря широкой популярности компьютеров, совместимых с ПК, и своей относительной стабильности на протяжении десятилетий. Он даже поддерживался операционными системами для других платформ. Иногда это происходило в дополнение к другим уже существовавшим или *кроссплатформенным* стандартам начальной загрузки и разбиения на разделы.^[3]

Однако записи разделов MBR и загрузочный код MBR, используемые в коммерческих операционных системах, ограничены 32 битами.^[1] Таким образом, максимальный размер диска, поддерживаемый схемой разметки MBR (без 32-битной арифметики) на дисках с 512-байтовыми секторами (как реальными, так и эмулируемыми), ограничен 2 TiB.^[1] Следовательно, для дисков большего размера, которые стали широко доступны с 2010 года, необходимо использовать другую схему разметки. Таким образом, схема разметки MBR постепенно заменяется на *таблицу разделов GUID* (GPT). Официальный подход обеспечивает целостность данных с помощью *защитной MBR*. В частности, он не обеспечивает обратную совместимость с операционными системами, которые не поддерживают схему GPT. Тем временем сторонние разработчики создали и внедрили множество форм *гибридных MBR*, чтобы поддерживать разделы, расположенные в первых физических 2 TiB диска, в обеих схемах разметки «параллельно» и/или чтобы старые операционные системы могли загружаться и с разделов GPT. Нынешний

нестандартный характер этих решений приводит к различным проблемам совместимости в определенных сценариях.

The MBR consists of 512 or more [bytes](#) located in the first [sector](#) of the drive.

It may contain one or more of:

- A [partition table](#) describing the partitions of a storage device. In this context the boot sector may also be called a *partition sector*.
- [Bootstrap code](#): Instructions to identify the configured bootable partition, then load and execute its [volume boot record](#) (VBR) as a [chain loader](#).
- Optional 32-bit *disk timestamp*.^[2]
- Optional 32-bit *disk signature*.^{[4][5][6][7]}

Разбиение диска

PC DOS IBM 2.0 представила утилиту [FDISK](#) для настройки и обслуживания разделов MBR. Если устройство хранения данных разбито на разделы в соответствии с этой схемой, его MBR содержит таблицу разделов, в которой указаны расположение, размер и другие атрибуты линейных областей, называемых разделами.

Сами разделы могут также содержать данные для описания более сложных схем разбиения на разделы, таких как [расширенные загрузочные записи](#) (EBR), [BSD disklabels](#) или разделы с метаданными [Logical Disk Manager](#).^[8]

MBR находится не в разделе, а в первом секторе устройства (физическое смещение 0), перед первым разделом. (Загрузочный сектор, присутствующий на устройстве без разделов или в отдельном разделе, называется [загрузочной записью тома](#).) В случаях, когда на компьютере используется [DDO BIOS overlay](#) или [менеджер загрузки](#), таблица разделов может быть перемещена в другое физическое расположение на устройстве. Например, [Ontrack Disk Manager](#) часто помещал копию исходного содержимого MBR во второй сектор, а затем скрывал себя от любой последующей загружаемой ОС или приложения, так что копия MBR воспринималась так, будто она по-прежнему находится в первом секторе.

Секторная схема

По соглашению, в схеме таблицы разделов MBR должно быть ровно четыре записи о первичных разделах, хотя некоторые операционные системы и системные инструменты расширили это число до пяти (Advanced Active Partitions (AAP) с [PTS-DOS 6.60](#)^[9] и [DR-DOS 7.07](#)), восемью ([AST](#) и [NEC MS-DOS 3.x](#)^{[10][11]} а также [Storage Dimensions SpeedStor](#)) или даже шестнадцать записей (с [Ontrack Disk Manager](#)).

Структура классической универсальной MBR

Адрес	Описание		Размер (байтов)
0x0000 (0)	Область кода начальной загрузки		446
0x01BE (446)	Partition entry №1	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №2		16
0x01DE (478)	Partition entry №3		16
0x01EE (494)	Partition entry №4		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 446 + 4×16 + 2			512

Structure of a *modern standard* MBR

Address	Description		Size (bytes)
0x0000 (0)	Bootstrap code area (part 1)		218
0x00DA (218)	0x0000	Disk timestamp ^{[2][b]} (optional; Windows 95B/98/98SE/ME (MS-DOS 7.1– 8.0). Alternatively, can serve as OEM loader signature with NEWLDR)	2
0x00DC (220)	Original physical drive (0x80 – 0xFF)		1
0x00DD (221)	Seconds (0–59)		1
0x00DE (222)	Minutes (0–59)		1
0x00DF (223)	Hours (0–23)		1
0x00E0 (224)	Bootstrap code area (part 2, code entry at 0x0000)		216 (or 222)
0x01B8 (440)	32-bit disk signature	Disk signature (optional; UEFI , Linux, Windows NT family and other OSes)	4
0x01BC (444)	0x0000 (0x5A5A if copy-protected)		2
0x01BE (446)	Partition entry №1	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №2		16
0x01DE (478)	Partition entry №3		16
0x01EE (494)	Partition entry №4		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 218 + 6 + 216 + 6 + 4×16 + 2			512

Structure of AAP MBR

Address	Description		Size (bytes)
0x0000 (0)	Bootstrap code area		428
0x01AC (428)	0x78	AAP signature (optional)	2
0x01AD (429)	0x56		
0x01AE (430)	AAP physical drive (0x80 – 0xFE ; 0x00 : not used; 0x01 – 0x7F , 0xFF : reserved)	AAP record (optional) (AAP partition entry #0 with special semantics)	1
0x01AF (431)	CHS (start) address of AAP partition/image file or VBR/EBR		3
0x01B2 (434)	Reserved for AAP partition type (0x00 if not used) (optional)		1
0x01B3 (435)	Reserved for CHS end address in AAP (optional; byte at offset 0x01B5 is also used for MBR checksum (PTS DE, BootWizard); 0x000000 if not used)		3
0x01B6 (438)	Start LBA of AAP image file or VBR/EBR or relative sectors of AAP partition (copied to offset +01C _{hex} in the loaded sector over the "hidden sectors" entry of a DOS 3.31 BPB (or emulation thereof)		4

	to also support EBR booting)		
0x01BA (442)	Reserved for sectors in AAP (optional; 0x00000000 if not used)		4
0x01BE (446)	Partition entry №1	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №2		16
0x01DE (478)	Partition entry №3		16
0x01EE (494)	Partition entry №4		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 428 + 2 + 16 + 4×16 + 2			512

Structure of NEWLDR MBR

Address	Description	Size (bytes)
0x0000 (0)	JMPS (<code>EB_{hex}</code>) / NEWLDR record size (often <code>0x0A</code> / <code>0x16</code> / <code>0x1C</code> for code start at <code>0x000C</code> / <code>0x0018</code> / <code>0x001E</code>)	2
0x0002 (2)	"NEWLDR" signature	6
0x0008 (8)	LOADER physical drive and boot flag (<code>0x80</code> – <code>0xFE</code> , <code>0x00</code> – <code>0x7E</code> , <code>0xFF</code> , <code>0x7F</code>) (if not used, this and following 3 bytes must be all 0)	1
0x0009 (9)	CHS address of LOADER boot sector or image file (f.e. <code>IBMBIO.LDR</code>) (<code>0x000000</code> if not used)	3
0x000C (12)	Allowed DL minimum, else take from partition table (<code>0x80</code> : default; <code>0x00</code> : always use DL; <code>0xFF</code> : always use table entry)	NEWLDR record (optional) 1
0x000D (13)	Reserved (default: <code>0x000000</code>)	3
0x0010 (16)	LBA of LOADER boot sector or image file (optional; <code>0x00000000</code> if not used)	4
0x0014 (20)	Patch offset of VBR boot unit (default <code>0x0000</code> if not used, else <code>0024_{hex}</code> or <code>01FD_{hex}</code>)	2
0x0016 (22)	Checksum (<code>0x0000</code> if not used)	2
0x0018 (24)	OEM loader signature ("MSWIN4" for <code>REAL/32</code> , see also offset <code>+0DA_{hex}</code> , corresponds with OEM label at offset <code>+003_{hex}</code> in VBRs (optional)	6
Varies	Bootstrap code area (code entry at <code>0x0000</code>)	Varies

0x01AC (428)	0x78	AAP signature (optional)	2
0x01AD (429)	0x56		
0x01AE (430)	AAP partition entry №0 with special semantics	AAP record (optional)	16
0x01BE (446)	Partition entry №1	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №2		16
0x01DE (478)	Partition entry №3		16
0x01EE (494)	Partition entry №4		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 30 + 398 + 2 + 16 + 4×16 + 2			512

Structure of AST/NEC MS-DOS and SpeedStor MBR

Address	Description		Size (bytes)
0x0000 (0)	Bootstrap code area		380
0x017C (380)	0x5A	AST/NEC signature (optional; not for SpeedStor)	2
0x017D (381)	0xA5		
0x017E (382)	Partition entry №8	AST/NEC expanded partition table (optional; also for SpeedStor)	16
0x018E (398)	Partition entry №7		16
0x019E (414)	Partition entry №6		16
0x01AE (430)	Partition entry №5		16
0x01BE (446)	Partition entry №4	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №3		16
0x01DE (478)	Partition entry №2		16
0x01EE (494)	Partition entry №1		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 380 + 2 + 4×16 + 4×16 + 2			512

Structure of Ontrack Disk Manager MBR

Address	Description		Size (bytes)
0x0000 (0)	Bootstrap code area		252
0x00FC (252)	0xAA	DM signature (optional)	2
0x00FD (253)	0x55		
0x00FE (254)	Partition entry	DM expanded partition table (optional)	16
0x010E (270)	Partition entry		16
0x011E (286)	Partition entry		16
0x012E (302)	Partition entry		16
0x013E (318)	Partition entry		16
0x014E (334)	Partition entry		16
0x015E (350)	Partition entry		16
0x016E (366)	Partition entry		16
0x017E (382)	Partition entry		16
0x018E (398)	Partition entry		16
0x019E (414)	Partition entry		16
0x01AE (430)	Partition entry		16
0x01BE (446)	Partition entry №1	Partition table (for primary partitions)	16
0x01CE (462)	Partition entry №2		16
0x01DE (478)	Partition entry №3		16

0x01EE (494)	Partition entry №4		16
0x01FE (510)	0x55	Boot signature ^[a]	2
0x01FF (511)	0xAA		
Total size: 252 + 2 + 12×16 + 4×16 + 2			512

Partition tables entries

An artifact of hard disk technology from the era of the [PC XT](#), the partition table subdivides a storage medium using units of [cylinders](#), [heads](#), and [sectors](#) ([CHS](#) addressing). These values no longer correspond to their namesakes in modern disk drives, as well as being irrelevant in other devices such as [solid-state drives](#), which do not physically have cylinders or heads.

In the CHS scheme, sector indices have (almost) always begun with sector 1 rather than sector 0 by convention, and due to an error in all versions of MS-DOS/PC DOS up to including 7.10, the number of heads is generally limited to 255^[h] instead of 256. When a CHS address is too large to fit into these fields, the [tuple](#) (1023, 254, 63) is typically used today, although on older systems, and with older disk tools, the cylinder value often wrapped around modulo the CHS barrier near 8 GB, causing ambiguity and risks of data corruption. (If the situation involves a "protective" MBR on a disk with a GPT, Intel's [Extensible Firmware Interface](#) specification requires that the tuple (1023, 255, 63) be used.) The 10-bit cylinder value is recorded within two bytes in order to facilitate making calls to the original/legacy [INT 13h](#) BIOS disk access routines, where 16 bits were divided into sector and cylinder parts, and not on byte boundaries.^[13]

Due to the limits of CHS addressing,^{[16][17]} a transition was made to using LBA, or [logical block addressing](#). Both the partition length and partition start address are sector values stored in the partition table entries as 32-bit quantities. The sector size used to be considered fixed at 512 (2⁹) bytes, and a broad range of important components including [chipsets](#), [boot sectors](#), [operating systems](#), [database engines](#), [partitioning](#) tools, [backup](#) and [file system](#) utilities and other software had this value hard-coded. Since the end of 2009, disk drives employing 4096-byte sectors ([4Kn](#) or [Advanced Format](#)) have been available, although the size of the sector for some of these drives was still reported as 512 bytes to the host system through conversion in the hard-drive firmware and referred to as 512 emulation drives ([512e](#)).

Since block addresses and sizes are stored in the partition table of an MBR using 32 bits, the maximum size, as well as the highest start address, of a partition using drives that have 512-byte sectors (actual or emulated) cannot exceed 2 [TiB](#)−512 bytes (2 199 023 255 040 bytes or 4 294 967 295 (2³²−1) sectors × 512 (2⁹) bytes per sector).^[1] Alleviating this capacity limitation was one of the prime motivations for the development of the GPT.

Since partitioning information is stored in the MBR partition table using a beginning block address and a length, it may in theory be possible to define partitions in such a way that the allocated space for a disk with 512-byte sectors gives a total size approaching 4 TiB, if all but one partition are located below the 2 TiB limit and the last one is assigned as starting at or close to block $2^{32}-1$ and specify the size as up to $2^{32}-1$, thereby defining a partition that requires 33 rather than 32 bits for the sector address to be accessed. However, in practice, only certain [LBA-48](#)-enabled operating systems, including Linux, FreeBSD and Windows 7^[18] that use 64-bit sector addresses internally actually support this. Due to code space constraints and the nature of the MBR partition table to only support 32 bits, boot sectors, even if enabled to support LBA-48 rather than [LBA-28](#), often use 32-bit calculations, unless they are specifically designed to support the full address range of LBA-48 or are intended to run on 64-bit platforms only. Any boot code or operating system using 32-bit sector addresses internally would cause addresses to wrap around accessing this partition and thereby result in serious data corruption over all partitions.

For disks that present a sector size other than 512 bytes, such as [USB external drives](#), there are limitations as well. A sector size of 4096 results in an eight-fold increase in the size of a partition that can be defined using MBR, allowing partitions up to 16 TiB ($2^{32} \times 4096$ bytes) in size.^[19] Versions of Windows more recent than Windows XP support the larger sector sizes, as well as Mac OS X, and [Linux](#) has supported larger sector sizes since 2.6.31^[20] or 2.6.32,^[21] but issues with boot loaders, partitioning tools and computer BIOS implementations present certain limitations,^[22] since they are often hard-wired to reserve only 512

Layout of one 16-byte partition entry^[12] (all multi-byte fields are [little-endian](#))

Offset (bytes)	Field length	Description																	
0x00	1 byte	Status or physical drive (bit 7 set is for active or bootable, old MBRs only accept 0x80, 0x00 means inactive, and 0x01–0x7F stand for invalid) ^[c]																	
0x01	3 bytes	CHS address of first absolute sector in partition. ^[d] The format is described by three bytes, see the next three rows.																	
0x01	1 byte	<table><tr><td colspan="8">h_{7–0}</td><td rowspan="2">head^[e]</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	h _{7–0}								head ^[e]	x	x	x	x	x	x	x	x
h _{7–0}								head ^[e]											
x	x	x	x	x	x	x	x												
0x02	1 byte	<table><tr><td colspan="2">C_{9–8}</td><td colspan="6">S_{5–0}</td><td rowspan="2">sector in bits 5–0; bits 7–6 are high bits of cylinder^[e]</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	C _{9–8}		S _{5–0}						sector in bits 5–0; bits 7–6 are high bits of cylinder ^[e]	x	x	x	x	x	x	x	x
C _{9–8}		S _{5–0}						sector in bits 5–0; bits 7–6 are high bits of cylinder ^[e]											
x	x	x	x	x	x	x	x												
0x03	1 byte	<table><tr><td colspan="8">C_{7–0}</td><td rowspan="2">bits 7–0 of cylinder^[e]</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	C _{7–0}								bits 7–0 of cylinder ^[e]	x	x	x	x	x	x	x	x
C _{7–0}								bits 7–0 of cylinder ^[e]											
x	x	x	x	x	x	x	x												
0x04	1 byte	Partition type ^[14]																	
0x05	3 bytes	CHS address of last absolute sector in partition. ^[d] The format is described by 3 bytes, see the next 3 rows.																	
0x05	1 byte	<table><tr><td colspan="8">h_{7–0}</td><td rowspan="2">head^[e]</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	h _{7–0}								head ^[e]	x	x	x	x	x	x	x	x
h _{7–0}								head ^[e]											
x	x	x	x	x	x	x	x												

bytes for sector buffers, causing memory to become overwritten for larger sector sizes. This may cause unpredictable behaviour as well, and therefore should be avoided when compatibility and standard conformity is an issue.

Where a data storage device has been partitioned with the GPT scheme, the master boot record will still contain a partition table, but its only purpose is to indicate the existence of the GPT and to prevent utility programs that understand only the MBR partition table scheme from creating any partitions in what they would otherwise see as free space on the disk, thereby accidentally erasing the GPT.

	0x06	1 byte	C₉₋₈ S₅₋₀ sector in bits 5-0; bits 7-6 are high bits of cylinder^[e]
	0x07	1 byte	C₇₋₀ bits 7-0 of cylinder
0x08		4 bytes	LBA of first absolute sector in the partition ^[f]
0x0C		4 bytes	Number of sectors in partition ^[g]

Начальная загрузка системы

На компьютерах [IBM PC-совместимыхзагрузочная прошивка](#) (содержащаяся в [ПЗУ BIOS](#)) загружает и выполняет главную загрузочную запись.^[23] В [PC/XT \(тип 5160\)](#) использовался [микропроцессор Intel 8088](#). Чтобы обеспечить совместимость, все системы с архитектурой BIOS x86 запускают микропроцессор в [операционном режиме](#), который называется [реальным режимом](#). BIOS считывает MBR с устройства хранения данных в [физическую память](#), а затем направляет микропроцессор к началу загрузочного кода. BIOS переводит процессор в реальный режим, после чего начинает выполнять программу MBR, поэтому предполагается, что в начале MBR находится [машинный код](#) реального режима.^[23]

Поскольку процедура начальной загрузки BIOS загружает и запускает ровно один сектор с физического диска, наличие таблицы разделов в MBR вместе с загрузочным кодом упрощает структуру программы MBR. Она содержит небольшую программу, которая загружает [загрузочную запись тома](#) (Volume Boot Record, VBR) целевого раздела. Затем управление передается этому коду, который отвечает за загрузку самой операционной системы. Этот процесс называется [цепной загрузкой](#).

Для загрузки [PC DOS](#) и [MS-DOS](#) были созданы популярные программы с кодом MBR, и аналогичный загрузочный код до сих пор широко используется. Эти загрузочные секторы предполагают использование схемы FDISK таблицы разделов и сканируют список разделов во встроенной таблице разделов MBR, чтобы найти единственный раздел,

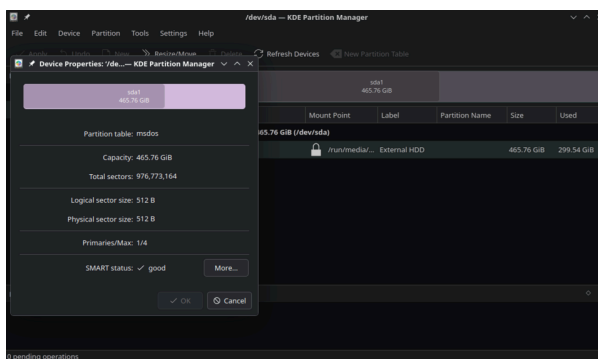
отмеченный *активным* *флагом*.^[24] Затем они загружают и запускают *загрузочную запись тома* (VBR) активного раздела.

Существуют альтернативные реализации загрузочного кода, некоторые из которых устанавливаются *загрузчиками*, работающими по-разному. Некоторые MBR-программы загружают дополнительный код загрузчика с первой дорожки диска, которую они считают «свободным» пространством, не выделенным ни для одного раздела диска, и запускают его. MBR-программа может взаимодействовать с пользователем, чтобы определить, какой раздел на каком диске следует загрузить, и может передать управление MBR другого диска. Другой код MBR содержит список расположений на диске (часто соответствующих содержимому *файлов* в *файловой системе*) оставшейся части кода загрузчика, которую необходимо загрузить и выполнить. (Первое зависит от особенностей, которые не являются универсальными для всех утилит для работы с разделами диска, особенно для тех, которые читают и записывают GPT. Второе требует, чтобы встроенный список расположений на диске обновлялся при внесении изменений, которые приводят к перемещению оставшейся части кода.)

On machines that do not use *x86* processors, or on x86 machines with non-BIOS firmware such as *Open Firmware* or *Extensible Firmware Interface* (EFI) firmware, this design is unsuitable, and the MBR is not used as part of the system bootstrap.^[25] EFI firmware is instead capable of directly understanding the GPT partitioning scheme and the *FAT* filesystem format, and loads and runs programs held as files in the *EFI System partition*.^[26] The MBR will be involved only insofar as it might contain a partition table for compatibility purposes if the GPT partition table scheme has been used.

There is some MBR replacement code that emulates EFI firmware's bootstrap, which makes non-EFI machines capable of booting from disks using the GPT partitioning scheme. It detects a GPT, places the processor in the correct operating mode, and loads the EFI compatible code from disk to complete this task.

Идентификация диска



Информация, содержащаяся в таблице разделов внешнего жесткого диска, как она отображается в утилите [KDE Partition Manager](#), работающей под управлением Linux

Помимо загрузочного кода и таблицы разделов, главные загрузочные записи могут содержать [сигнатуру диска](#). Это 32-битное значение, предназначенное для уникальной идентификации носителя (в отличие от дискового устройства — эти два понятия не обязательно совпадают для съемных жестких дисков).

Дисковая подпись была введена в Windows NT версии 3.5, но сейчас она используется в нескольких операционных системах, включая [ядро Linux](#) версии 2.6 и более поздних. Инструменты Linux могут использовать дисковую подпись NT для определения того, с какого диска загрузилась машина. ^[27]

Windows NT (и более поздние операционные системы Microsoft) использует сигнатуру диска в качестве индекса для всех разделов на любом диске, когда-либо подключенном к компьютеру под управлением этой ОС. Эти сигнатуры хранятся в ключах [реестра Windows](#), в основном для сохранения постоянных сопоставлений между разделами диска и буквами дисков. Она также может использоваться в файлах [BOOT.INI](#) Windows NT (хотя в большинстве случаев этого не происходит) для описания расположения загрузочных разделов Windows NT (или более поздних версий). ^[28] Один из ключей (среди многих), в котором в реестре Windows 2000/XP отображаются подписи дисков NT, выглядит так:

```
HKEY_LOCAL_MACHINE\SYSTEM\MountedDevices\
```

Если подпись диска, хранящаяся в MBR, была A8 E1 B9 D2 (в таком порядке), а его первый раздел соответствовал логическому диску C: в Windows, то `REG_BINARY` данных под значением ключа `\DosDevices\C:` выглядели бы так:

```
A8 E1 B9 D2 00 7E 00 00 00 00 00 00
```

The first four bytes are said disk signature. (In other keys, these bytes may appear in reverse order from that found in the MBR sector.) These are followed by eight more bytes, forming a 64-bit integer, in [little-endian](#) notation, which are used to locate the byte offset of this partition. In this case, 00 7E corresponds to the hexadecimal value 0x7E00 (32,256). Under the assumption that the drive in question reports a sector size of 512 bytes, then dividing this byte offset by 512 results in 63, which is the physical sector number (or LBA) containing the first sector of the partition (unlike the *sector count* used in the sectors value of CHS tuples, which counts from **one**, the absolute or LBA sector value starts [counting from zero](#)).

If this disk had another partition with the values 00 F8 93 71 02 following the disk signature (under, e.g., the key value `\DosDevices\D:`), it would begin at byte offset 0x00027193F800 (10,495,457,280), which is also the first byte of physical sector 20,498,940.

Starting with [Windows Vista](#), the disk signature is also stored in the [Boot Configuration Data](#) (BCD) store, and the boot process depends on it.^[29] If the disk signature changes, cannot be found or has a conflict, Windows is unable to boot.^[30] Unless Windows is forced to use the overlapping part of the LBA address of the Advanced Active Partition entry as pseudo-disk signature, Windows' usage is conflictive with the Advanced Active Partition feature of PTS-DOS 7 and DR-DOS 7.07, in particular if their boot code is located outside the first 8 GB of the disk, so that LBA addressing must be used.

Рекомендации по программированию

MBR появилась в [PC XT](#).^[31] [IBM PC-совместимые](#) компьютеры имеют [обратный порядок байтов](#), то есть [процессор](#) хранит числовые значения, занимающие два или более байта, в памяти [наименее значимым байтом](#) вперед. Формат MBR на носителе отражает это соглашение. Таким образом, сигнатура MBR будет отображаться в [редакторе дисков](#) как последовательность `55 AA`.^[a]

Начальная последовательность загрузки в BIOS загружает первую найденную корректную MBR в [физическую память](#) компьютера по [адресу](#) 0x7C00 в 0x7DFF.^[31] Последняя инструкция, выполняемая в коде BIOS, будет «переходом» по этому адресу, чтобы сразу перейти к началу копии MBR. Основной проверкой для большинства BIOS является подпись в области 0x01FE, хотя разработчик BIOS может включить и другие проверки, например убедиться, что MBR содержит корректную таблицу разделов без записей, указывающих на сектора, выходящие за пределы заявленной емкости диска.

Для BIOS съемные (например, дискеты) и жесткие диски практически идентичны. В обоих случаях BIOS считывает первый физический сектор носителя в оперативную память по абсолютному адресу 0x7C00, проверяет подпись в последних двух байтах загруженного сектора, а затем, если подпись верна, передает управление первому байту сектора с помощью инструкции перехода (JMP). Единственное реальное отличие, которое делает BIOS, заключается в том, что (по умолчанию или если порядок загрузки не настраивается) он пытается загрузиться с первого съемного диска, прежде чем пытаться загрузиться с первого жесткого диска. С точки зрения BIOS, загрузка MBR, которая считывает загрузочную запись тома в оперативную память, ничем не отличается от загрузки загрузочной записи гибкого диска, которая считывает в оперативную память объектный код загрузчика операционной системы. В обоих случаях программа, загруженная BIOS, выполняет последовательную загрузку операционной системы.

While the MBR [boot sector](#) code expects to be loaded at physical address 0x0000:0x7C00,^[i] all the memory from physical address 0x0000:0x0501 (address 0x0000:0x0500 is the last one used by a Phoenix BIOS)^[13] to 0x0000:0x7FFF,^[31] later relaxed to 0x0000:0xFFFF^[32] (and sometimes^[j] up to 0x9000:0xFFFF)—the end of the first 640 ~~KB~~—is available in real mode.^[k] The `INT 12h` [BIOS interrupt call](#) may help in determining how much memory can be allocated safely (by default, it simply reads the base memory size in KB from [segment:offset](#) location 0x0040:0x0013, but it may be hooked by other resident pre-boot software like BIOS overlays, [RPL](#) code or viruses to reduce the reported amount of available memory in order to keep other boot stage software like boot sectors from overwriting them).

The last 66 bytes of the 512-byte MBR are reserved for the partition table and other information, so the MBR boot sector program must be small enough to fit within 446 bytes of memory or less.

The MBR code examines the partition table, selects a suitable partition and loads the program that will perform the next stage of the boot process, usually by making use of INT 13h [BIOS calls](#). The MBR bootstrap code loads and runs (a boot loader- or operating system-dependent) [volume boot record](#) code that is located at the beginning of the "active" partition. The volume boot record will fit within a 512-byte sector, but it is safe for the MBR code to load additional sectors to accommodate boot loaders longer than one sector, provided they do not make any assumptions on what the sector size is. In fact, at least 1 KB of RAM is available at address 0x7C00 in every IBM XT- and AT-class machine, so a 1 KB sector could be used with no problem. Like the MBR, a volume boot record normally expects to be loaded at address 0x0000:0x7C00. This derives from the fact that the volume boot record design originated on unpartitioned media, where a volume boot record would be directly loaded by the BIOS boot procedure; as mentioned above, the BIOS treats MBRs and volume boot records (VBRs)^[l] exactly alike. Since this is the same location where the MBR is loaded, one of the first tasks of an MBR is to [relocate](#) itself somewhere else in memory. The relocation address is determined by the MBR, but it is most

often 0x0000:0x0600 (for MS-DOS/PC DOS, OS/2 and Windows MBR code) or 0x0060:0x0000 (most DR-DOS MBRs). (Even though both of these segmented addresses resolve to the same physical memory address in real mode, for [Apple Darwin](#) to boot, the MBR must be relocated to 0x0000:0x0600 instead of 0x0060:0x0000, since the code depends on the DS:SI pointer to the partition entry provided by the MBR, but it erroneously refers to it via 0x0000:SI only.^[33]) It is important not to relocate to other addresses in memory because many [VBRs](#) will assume a certain standard memory layout when loading their boot file.

The *Status* field in a partition table record is used to indicate an active partition. Standard-conformant MBRs will allow only one partition marked active and use this as part of a sanity-check to determine the existence of a valid partition table. They will display an error message, if more than one partition has been marked active. Some non-standard MBRs will not treat this as an error condition and just use the first marked partition in the row.

Traditionally, values other than 0x00 (not active) and 0x80 (active) were invalid and the bootstrap program would display an error message upon encountering them. However, the [Plug and Play BIOS Specification](#) and [BIOS Boot Specification](#) (BBS) allowed other devices to become bootable as well since 1994.^{[32][34]} Consequently, with the introduction of MS-DOS 7.10 (Windows 95B) and higher, the MBR started to treat a set bit 7 as active flag and showed an error message for values 0x01..0x7F only. It continued to treat the entry as physical drive unit to be used when loading the corresponding partition's VBR later on, thereby now also accepting other boot drives than 0x80 as valid, however, MS-DOS did not make use of this extension by itself. Storing the actual physical drive number in the partition table does not normally cause backward compatibility problems, since the value will differ from 0x80 *only* on drives other than the first one (which have not been bootable before, anyway). However, even with systems enabled to boot off other drives, the extension may still not work universally, for example, after the BIOS assignment of physical drives has changed when drives are removed, added or swapped. Therefore, per the [BIOS Boot Specification](#) (BBS),^[32] it is best practice for a modern MBR accepting bit 7 as active flag to pass on the DL value originally provided by the BIOS instead of using the entry in the partition table.

BIOS to MBR interface

The MBR is loaded at memory location 0x0000:0x7C00 and with the following [CPU](#) registers set up when the prior bootstrap loader (normally the [IPL](#) in the BIOS) passes execution to it by jumping to 0x0000:0x7C00 in the CPU's [real mode](#).

- [CS:IP](#) = 0x0000:0x7C00 (fixed)

Some Compaq BIOSes erroneously use 0x07C0:0x0000 instead. While this resolves to the same location in real mode memory, it is non-standard and should be avoided, since MBR code assuming certain register values or not written to be relocatable may not work otherwise.

- **DL** = boot drive unit ([fixed disks](#) / [removable drives](#): 0x80 = first, 0x81 = second, ..., 0xFE; [floppies](#) / [superfloppies](#): 0x00 = first, 0x01 = second, ..., 0x7E; values 0x7F and 0xFF are reserved for ROM / remote drives and must not be used on disk).^{[35][36]}

DL is supported by IBM BIOSes as well as most other BIOSes. The Toshiba T1000 BIOS is known not to support this properly, and some old Wyse 286 BIOSes use DL values greater or equal to 2 for fixed disks (thereby reflecting the logical drive numbers under DOS rather than the physical drive numbers of the BIOS). USB sticks configured as removable drives typically get an assignment of DL = 0x80, 0x81, etc. However, some rare BIOSes erroneously presented them under DL = 0x01, just as if they were configured as superfloppies.

A standard conformant BIOS assigns numbers greater or equal to 0x80 exclusively to fixed disk / removable drives, and traditionally only values 0x80 and 0x00 were passed on as physical drive units during boot. By convention, only fixed disks / removable drives are partitioned, therefore, the only DL value a MBR could see traditionally was 0x80. Many MBRs were written to ignore the DL value and work with a hard-wired value (normally 0x80), anyway. The [Plug and Play BIOS Specification](#) and [BIOS Boot Specification](#) (BBS) allow other devices to become bootable as well since 1994.^{[32][34]} The later recommends that MBR and VBR code should use DL rather than internally hardwired defaults.^[32] This will also ensure compatibility with various non-standard assignments (see examples above), as far as the MBR code is concerned.

Bootable CD-ROMs following the [El Torito](#) specification may contain disk images mounted by the BIOS to occur as floppy or superfloppies on this interface. DL values of 0x00 and 0x01 may also be used by [Protected Area Run Time Interface Extension Services](#) (PARTIES) and [Trusted Computing Group](#) (TCG) BIOS extensions in Trusted mode to access otherwise invisible PARTIES partitions, disk image files located via the [Boot Engineering Extension Record](#) (BEER) in the last physical sector of a hard disk's [Host Protected Area](#) (HPA). While designed to emulate floppies or superfloppies, MBR code accepting these non-standard DL values allows to use images of partitioned media at least in the boot stage of operating systems.

- **DH** bit 5 = 0: device supported through [INT 13h](#); else: don't care (should be zero). DH is supported by some IBM BIOSes.
- Some of the other registers may typically also hold certain register values (DS, ES, SS = 0x0000; SP = 0x0400) with original IBM ROM BIOSes, but this is nothing to rely on, as other BIOSes may use other values. For this reason, MBR code by IBM, Microsoft, Digital Research, etc. never did take any advantage of it. Relying on these register values in boot sectors may also cause problems in chain-boot scenarios.

Systems with [Plug-and-Play](#) BIOS or BBS support will provide a pointer to PnP data in addition to DL.^{[32][34]}

- **DL** = boot drive unit (see above)

- **ES:DI** = points to "`$PnP`" installation check structure

This information allows the boot loader in the MBR (or VBR, if passed on) to actively interact with the BIOS or a resident PnP / BBS BIOS overlay in memory in order to configure the boot order, etc., however, this information is ignored by most standard MBRs and VBRs. Ideally, ES:DI is passed on to the VBR for later use by the loaded operating system, but PnP-enabled operating systems typically also have fallback methods to retrieve the PnP BIOS entry point later on so that most operating systems do not rely on this.

MBR to VBR interface

By convention, a standard conformant MBR passes execution to a successfully loaded VBR, loaded at memory location 0x0000:0x7C00, by jumping to 0x0000:0x7C00 in the CPU's real mode with the following registers maintained or specifically set up:

- CS:IP = 0x0000:0x7C00^[m] (constant)
- DL = boot drive unit (see above)

MS-DOS 2.0–7.0 / PC DOS 2.0–6.3 MBRs do not pass on the DL value received on entry, but they rather use the boot status entry in the partition table entry of the selected primary partition as physical boot drive unit. Since this is, by convention, 0x80 in most MBR partition tables, it won't change things unless the BIOS attempted to boot off a physical device other than the first fixed disk / removable drive in the row. This is also the reason why these operating systems cannot boot off a second hard disk, etc. Some FDISK tools allow to mark partitions on secondary disks as "active" as well. In this situation, knowing that these operating systems cannot boot off other drives anyway, some of them continue to use the traditionally fixed value of 0x80 as active marker, whereas others use values corresponding with the currently assigned physical drive unit (0x81, 0x82), thereby allowing booting from other drives, at least in theory. In fact, this will work with many MBR codes, which take a set bit 7 of the boot status entry as active flag rather than insisting on 0x80, however, MS-DOS/PC DOS MBRs are hard-wired to accept the fixed value of 0x80 only. Storing the actual physical drive number in the partition table will also cause problems, when the BIOS assignment of physical drives changes, for example when drives are removed, added or swapped. Therefore, for a normal MBR accepting bit 7 as active flag and otherwise just using and passing on to the VBR the DL value originally provided by the BIOS allows for maximum flexibility. MS-DOS 7.1–8.0 MBRs have changed to treat bit 7 as active flag and any values 0x01..0x7F as invalid, but they still take the physical drive unit from the partition table rather than using the DL value provided by the BIOS. DR-DOS 7.07 extended MBRs treat bit 7 as active flag and use and pass on the BIOS DL value by default (including non-standard values 0x00..0x01 used by some BIOSes also for partitioned media), but they also provide a special **NEWLDR** configuration block in order to support alternative boot methods in conjunction with **LOADER** and **REAL/32** as well as to change the detail behaviour of the MBR, so that it can also work with drive values retrieved

from the partition table (important in conjunction with LOADER and AAPs, see NEWLDR offset `0x000C`), translate Wyse non-standard drive units 0x02..0x7F to 0x80..0xFD, and optionally fix up the drive value (stored at offset `0x019` in the [Extended BIOS Parameter Block](#) (EBPB) or at sector offset `0x01FD`) in loaded VBRs before passing execution to them (see NEWLDR offset `0x0014`)—this also allows other boot loaders to use NEWLDR as a chain-loader, configure its in-memory image on the fly and "tunnel" the loading of VBRs, EBRs, or AAPs through NEWLDR.

- The contents of DH and ES:DI should be preserved by the MBR for full Plug-and-Play support (see above), however, many MBRs, including those of MS-DOS 2.0–8.0 / PC DOS 2.0–6.3 and Windows NT/2000/XP, do not. (This is unsurprising, since those versions of DOS predate the Plug-and-Play BIOS standard, and previous standards and conventions indicated no requirements to preserve any register other than DL.) Some MBRs set DH to 0.

The MBR code passes additional information to the VBR in many implementations:

- DS:SI = points to the 16-byte [MBR partition table](#) entry (in the relocated MBR) corresponding with the activated VBR. [PC-MOS](#) 5.1 depends on this to boot if no partition in the partition table is flagged as bootable. In conjunction with LOADER, [Multiuser DOS](#) and [REAL/32](#) boot sectors use this to locate the boot sector of the active partition (or another bootstrap loader like [IBMBIO.LDR](#) at a fixed position on disk) if the boot file (LOADER.SYS) could not be found. [PTS-DOS](#) 6.6 and [S/DOS](#) 1.0 use this in conjunction with their [Advanced Active Partition](#) (AAP) feature. In addition to support for LOADER and AAPs, DR-DOS 7.07 can use this to determine the necessary INT 13h access method when using its dual CHS/LBA VBR code and it will update the boot drive / status flag field in the partition entry according to the effectively used DL value. [Darwin](#) bootloaders (Apple's `boot1h`, `boot1u`, and David Elliott's `boot1fat32`) depend on this pointer as well, but additionally they don't use DS, but assume it to be set to 0x0000 instead.^[33] This will cause problems if this assumption is incorrect. The MBR code of OS/2, MS-DOS 2.0 to 8.0, PC DOS 2.0 to 7.10 and Windows NT/2000/XP provides this same interface as well, although these systems do not use it. The Windows Vista/7 MBRs no longer provide this DS:SI pointer. While some extensions only depend on the 16-byte partition table entry itself, other extensions may require the whole 4 (or 5 entry) partition table to be present as well.
- DS:BP = optionally points to the 16-byte [MBR partition table](#) entry (in the relocated MBR) corresponding with the activated VBR. This is identical to the pointer provided by DS:SI (see above) and is provided by MS-DOS 2.0–8.0, PC DOS 2.0–7.10, Windows NT/2000/XP/Vista/7 MBRs. It is, however, not supported by most third-party MBRs.

Under DR-DOS 7.07 an extended interface may be optionally provided by the extended MBR and in conjunction with LOADER:

- [AX](#) = magic signature indicating the presence of this NEWLDR extension (0x0EDC)

- DL = boot drive unit (see above)
- DS:SI = points to the 16-byte [MBR partition table](#) entry used (see above)
- ES:BX = start of boot sector or NEWLDR sector image (typically 0x7C00)
- CX = reserved

In conjunction with GPT, an [Enhanced Disk Drive Specification](#) (EDD) 4 [Hybrid MBR](#) proposal recommends another extension to the interface:^[37]

- EAX = 0x54504721 ("!GPT")
- DL = boot drive unit (see above)
- DS:SI = points to a Hybrid MBR handover structure, consisting of a 16-byte dummy [MBR partition table](#) entry (with all bits set except for the boot flag at offset 0x00 and the [partition type](#) at offset 0x04) followed by additional data. This is partially compatible with the older DS:SI extension discussed above, if only the 16-byte partition entry, not the whole partition table is required by these older extensions.

Since older operating systems (including their VBRs) do not support this extension nor are they able to address sectors beyond the 2 TiB barrier, a GPT-enabled hybrid boot loader should still emulate the 16-byte dummy MBR partition table entry if the boot partition is located within the first 2 TiB.^[n]

- ES:DI = points to "\$PnP" installation check structure (see above)

Редактирование и замена содержимого

Хотя можно напрямую манипулировать [байтами](#) в секторе MBR с помощью различных [дисковых редакторов](#), существуют инструменты для записи в MBR фиксированных наборов работающего кода. Начиная с версии MS-DOS 5.0, в программе `FDISK` есть переключатель `/MBR`, который позволяет перезаписать код MBR.^[38] В [Windows 2000](#) и [Windows XP](#) для записи нового кода MBR на устройство хранения данных можно использовать [консоль восстановления](#) с помощью команды `fixmbr`. В [Windows Vista](#) и [Windows 7](#) можно использовать [среду восстановления](#) для записи нового кода MBR с помощью команды `BOOTREC /FIXMBR`. Некоторые сторонние утилиты также позволяют напрямую редактировать содержимое таблиц разделов (без необходимости разбираться в шестнадцатеричном формате или редакторах дисков/секторов), например MBRWizard.^[o]

`dd` Это команда POSIX, которая обычно используется для чтения или записи данных в любое место на устройстве хранения, включая MBR. В [Linux](#) для установки MBR Windows можно использовать `ms-sys`. В проектах [GRUB](#) и [LILO](#) есть инструменты для записи кода в сектор MBR, а именно `grub-install` и `lilo -mbr`. Интерактивная консоль GRUB

Legacy может записывать данные в MBR с помощью команд `setup` и `embed`. Однако в GRUB2 в настоящее время требуется, чтобы `grub-install` запускалась из операционной системы.

Различные программы могут создавать "резервные копии" как основной таблицы разделов, так и логических разделов в расширенном разделе.

Linux `sfdisk` (на [SystemRescueCD](#)) может сохранять резервную копию основной и расширенной таблиц разделов. Она создает файл, который можно открыть в текстовом редакторе или использовать с помощью `sfdisk` для восстановления основной/расширенной таблицы разделов. Пример команды для резервного копирования таблицы разделов: `sfdisk -d /dev/hda > hda.out`, а для восстановления — `sfdisk /dev/hda < hda.out`. Таким способом можно скопировать таблицу разделов с одного диска на другой, что полезно при настройке зеркалирования, но `sfdisk` выполняет команду без запросов и предупреждений, используя `sfdisk -d /dev/sda | sfdisk /dev/sdb`.^[39]

Смотрите также

- [Расширенная загрузочная запись](#) (EBR)
- [Загрузочная запись тома](#) (VBR)
- [Таблица разделов GUID](#) (GPT)
- [Загрузочный раздел BIOS](#)
- [Системный раздел EFI](#)
- [Запись о расширении программы загрузки](#) (BEER)
- [Защищенная зона хоста](#) (HPA)
- [Наложение конфигурации устройства](#) (DCO)
- [Карта разделов Apple](#) (APM)
- [Блок жесткого диска Amiga](#) (RDB)
- [Оглавление тома](#) (VTOC)
- [BSD disklabel](#)
- [Загрузчик](#)
- [Клонирующий диск восстановления](#)

- [с разделением GNU](#)
- [по разделам](#)

Примечания

- a. Сигнатура по смещению 0x01FE в загрузочных секторах — `55hex AAhex`, то есть 0x55 по смещению 0x01FE и `AAhex` по смещению 0x01FF. Поскольку в контексте машин, совместимых с **IBM PC**, должно предполагаться **младшее порядковое** представление, это может быть записано как 16-разрядное слово `AA55hex` в программах для процессоров **x86** (обратите внимание на порядок замены), тогда как в программах для других архитектур ЦП оно должно быть записано как `55AAhex`, использующих **старшее порядковое** представление. Поскольку это неоднократно перепутывалось в книгах и даже в оригинальных справочных документах Microsoft, в этой статье используется представление на диске на основе смещения по байтам, чтобы избежать любого возможного неправильного толкования.
- b. Для обеспечения целостности кода загрузчика MBR важно, чтобы байты по адресу 0x00DA были равны 0x00DF никогда не изменяются, за исключением случаев, когда все шесть байтов имеют значение 0 или когда одновременно заменяется весь код загрузчика MBR (кроме (расширенной) таблицы разделов). Это включает в себя сброс этих значений до `00 00 00 00 00 00hex`, если только код, хранящийся в MBR, не известен. Windows придерживается этого правила.
- c. Изначально значения статуса, отличные от 0x00 и 0x80 были недопустимыми значениями, но современные MBR рассматривают бит 7 как активный флаг и используют эту запись для хранения физического загрузочного блока.
- d. Поля начального сектора ограничены 1023+1 цилиндрами, 255+1 головками и 63 секторами; поля конечного сектора имеют те же ограничения.
- e. Диапазон секторов — от 1 до 63; диапазон цилиндров — от 0 до 1023; диапазон головок — от 0 до 255 включительно.^[13]
- f. Эта запись используется операционными системами при определенных обстоятельствах; в таких случаях адреса CHS игнорируются.^[15]
- g. Ноль зарезервирован и не должен использоваться в обычных записях о разделах. Эта запись используется операционными системами при определенных обстоятельствах; в таких случаях адреса CHS игнорируются.^[15]

- h. "Цитата: [В большинстве] версий MS-DOS (включая MS-DOS 7 [Windows 95]) есть ошибка, которая не позволяет загружаться с жестких дисков с 256 головками (FFh), поэтому многие современные BIOS поддерживают отображение максимум 255 головок (FEh)." [RBIL^{\[40\]\[41\]}](#)
- i. Адрес `0000_hex:7C00_hex` — это первый байт 32-го килобайта оперативной памяти. Загрузка загрузочной программы по этому адресу исторически была причиной того, что минимальный объем оперативной памяти оригинального IBM PC (тип 5150) составлял 16 КБ, а для дисковой опции IBM XT требовалось 32 КБ.
- j. If there is an **EBDA**, the available memory ends below it.
- k. Very old machines may have less than 640 KB (`A0000_hex` or 655,360 bytes) of memory. In theory, only 32 KB (up to `0000_hex:7FFF_hex`) or 64 KB (up to `0000_hex:FFFF_hex`) are guaranteed to exist; this would be the case on an IBM XT-class machine equipped with only the required minimum amount of memory for a disk system.
- l. This applies when the BIOS handles a VBR, which is when it is in the first physical sector of unpartitioned media. Otherwise, the BIOS has nothing to do with the VBR. The design of VBRs is such as it is because VBRs originated solely on unpartitioned floppy disk media—the type 5150 IBM PC originally had no hard disk option—and the partitioning system using an MBR was later developed as an adaptation to put more than one volume, each beginning with its own VBR as-already-defined, onto a single fixed disk. By this design, essentially the MBR emulates the BIOS boot routine, doing the same things the BIOS would do to process this VBR and set up the initial operating environment for it just as if the BIOS had found that VBR on an unpartitioned medium.
- m. IP is set as a result of the jump. CS may be set to 0 either by executing a far jump or by loading the register value explicitly before executing a near jump. (It is impossible for jumped-to x86 code to detect whether a near or far jump was used to reach it [unless the code that made the jump separately passes this information in some way].)
- n. This is not part of the above mentioned proposal, but a natural consequence of pre-existing conditions.
- o. For example, *PowerQuest's Partition Table Editor* (PTEDIT32.EXE), which runs under Windows operating systems, is still available here: [Symantec's FTP site \(ftp://ftp.symantec.com/public/english_us_canada/tools/pq/utilities/PTEDIT32.zip\)](ftp://ftp.symantec.com/public/english_us_canada/tools/pq/utilities/PTEDIT32.zip) .

ССЫЛКИ

1. "Поддержка Windows для жестких дисков объемом более 2 ТБ" (<http://support.microsoft.com/kb/2581408>) . 1. Microsoft. 26 июня 2013 г. 2581408. Архивировано (<https://web.archive.org/web/20170427084734/https://support.microsoft.com/en-us/help/2581408/>

[windows-support-for-hard-disks-that-are-larger-than-2-tb](#)) из оригинала 27 апреля 2017 г.. Дата обращения: 28 августа 2013 г..

2. Седори, Дэниел Б. (2004). "Загадочные байты (или байты диска/временной метки) главной загрузочной записи (MBR) MS-Windows 95B, 98, 98SE и Me" (<http://daniel.sedory.com/asm/mbr/mystery.htm>) . Главные загрузочные записи. daniel.sedory.com. Архивировано (<https://web.archive.org/web/20170824002828/http://thestarman.pcministry.com/asm/mbr/mystery.htm>) из оригинала 24.08.2017. Дата обращения: 27 июля 2026 года.
3. Лукас, Майкл (2003). *Absolute OpenBSD: Unix для практичных параноиков* (<https://books.google.com/books?id=wuUuTXMkNx8C&pg=PA72>) . No Starch Press. стр. 73. ISBN 9781886411999. Дата обращения: 9 апреля 2011. "В каждой операционной системе есть инструменты для управления MBR-разделами. К сожалению, в каждой операционной системе MBR-разделы обрабатываются немного по-разному."
4. Нортон, Питер; Кларк, Скотт (2002). «Что нового внутри ПК» Питера Нортон (<https://archive.org/details/peternortonsnewi00pete/page/360>) . Sams Publishing. стр. 360–361 (<https://archive.org/details/peternortonsnewi00pete/page/360>) . ISBN 0-672-32289-7.
5. Грейвс, Майкл У. (2004). *Руководство по обслуживанию и ремонту компьютерного оборудования*. Thomson Delmar. стр. 276. ISBN 1-4018-5230-0.
6. Эндрюс, Джин (2003). «Модернизация и ремонт» с Джин Эндрюс (<https://archive.org/details/upgraderepairwit0000andr/page/646>) . Thomson Course Technology. стр. 646 (<https://archive.org/details/upgraderepairwit0000andr/page/646>) . ISBN 1-59200-112-2.
7. Boswell, William (2003). *Inside Windows Server 2003*. Addison-Wesley Professional. p. 13. ISBN 0-7357-1158-5.
8. Smith, Roderick W. (2000). *The Multi-Boot Configuration Handbook* (https://archive.org/details/isbn_9780789722836/page/260) . Que Publishing. pp. 260–261 (https://archive.org/details/isbn_9780789722836/page/260) . ISBN 0-7897-2283-6.
9. Brouwer, Andries Evert (2004-04-22) [2000]. "Properties of partition tables" (http://www.win.tue.nl/~aeb/partitions/partition_types-2.html) . *Partition types*. Archived (https://web.archive.org/web/20170824112413/http://www.win.tue.nl/~aeb/partitions/partition_types-2.html) from the original on 2017-08-24. Retrieved 2017-08-24. "Matthias [R.] Paul writes: "[...] PTS-DOS [uses] a special fifth partition entry in front of the other four entries in the MBR and corresponding AAP-aware MBR bootstrap code. [...]" "

10. Brouwer, Andries Evert (2004-04-22) [2000]. "Properties of partition tables" (http://www.win.tue.nl/~aeb/partitions/partition_types-2.html) . *Partition types*. Archived (https://web.archive.org/web/20170824112413/http://www.win.tue.nl/~aeb/partitions/partition_types-2.html) from the original on 2017-08-24. Retrieved 2017-08-24. "Some OEM systems, such as AST DOS (type `14hex`) and NEC DOS (type `24hex`) had 8 instead of 4 partition entries in their MBR sectors. (Matthias R. Paul)." (NB. NEC MS-DOS 3.30 and AST MS-DOS partition tables with eight entries are preceded with a signature `A55Ahex` at offset 0x017C.)
11. Sedory, Daniel B. (2007-05-18) [2003]. "Notes on the Differences in one OEM version of the DOS 3.30 MBR" (<http://daniel.sedory.com/asm/mbr/DOS33MBR.htm>) . *Master Boot Records*. Archived (<https://web.archive.org/web/20170824003525/http://thestarman.pcministry.com/asm/mbr/DOS33MBR.htm>) from the original on 2017-08-24. Retrieved 2026-07-27. "When we added partitions to this NEC table, the first one was placed at offsets 0x01EE through 0x01FD and the next entry was added just above it. So, the entries are inserted and listed backwards from that of a normal Table. Thus, looking at such a Table with a disk editor or partition listing utility, it would show the first entry in a NEC eight-entry table as being the last one (fourth entry) in a normal Partition Table." (NB. Shows an 8-entry partition table and where its boot code differs from MS-DOS 3.30.)
12. "Partition Table" (http://wiki.osdev.org/Partition_Table) . osdev.org. 2017-03-18 [2007-03-06]. Archived (https://web.archive.org/web/20170824123235/http://wiki.osdev.org/Partition_Table) from the original on 2017-08-24. Retrieved 2017-08-24.
13. *System BIOS for IBM PC/XT/AT Computers and Compatibles*. Phoenix technical reference. Addison-Wesley. 1989. ISBN 0-201-51806-6.
14. Brouwer, Andries Evert (2013) [1995]. "List of partition identifiers for PCs" (http://www.win.tue.nl/~aeb/partitions/partition_types-1.html) . *Partition types*. Archived (https://web.archive.org/web/20170824112750/http://www.win.tue.nl/~aeb/partitions/partition_types-1.html) from the original on 2017-08-24. Retrieved 2017-08-24.
15. Wood, Sybil (2002). *Microsoft Windows 2000 Server Operations Guide* (<https://archive.org/details/microsoftwindows00wood/page/18>) . Microsoft Press. p. 18 (<https://archive.org/details/microsoftwindows00wood/page/18>) . ISBN 978-0-73561796-4.
16. "An Introduction to Hard Disk Geometry" (<https://web.archive.org/web/20130204205616/http://www.tech-juice.org/2011/08/08/an-introduction-to-hard-disk-geometry/>) . *Tech Juice*. 2012-12-06 [2011-08-08]. Archived from the original (<http://www.tech-juice.org/2011/08/08/an-introduction-to-hard-disk-geometry/>) on 2013-02-04.

17. Kozierok, Charles M. (2001-04-17). "BIOS and the Hard Disk" (<http://www.pcguides.com/ref/hdd/bios/bios.htm>) . *The PC Guide*. Archived (<https://web.archive.org/web/20170617080653/http://www.pcguides.com/ref/hdd/bios/bios.htm>) from the original on 2017-06-17. Retrieved 2013-04-19.
18. Smith, Robert (2011-06-26). "Working Around MBR's Limitations" (<http://www.rodsbooks.com/gdisk/workarounds.html>) . *GPT fdisk Tutorial*. Archived (<https://web.archive.org/web/20170824114940/http://www.rodsbooks.com/gdisk/workarounds.html>) from the original on 2017-08-24. Retrieved 2013-04-20.
19. "More than 2 TiB on a MBR disk" (<http://superuser.com/questions/562331/mbr-partition-with-more-than-2-tb>) . superuser.com. 2013-03-07. Archived (<https://web.archive.org/web/20170824122749/https://superuser.com/questions/562331/mbr-partition-with-more-than-2-tb>) from the original on 2017-08-24. Retrieved 2013-10-22.
20. "Transition to Advanced Format 4K Sector Hard Drives" (<http://www.seagate.com/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/>) . *Tech Insight*. Seagate Technology. 2012. Archived (<https://web.archive.org/web/20170824194910/http://www.seagate.com/de/de/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/>) from the original on 2017-08-24. Retrieved 2013-04-19.
21. Calvert, Kelvin (2011-03-16). "WD AV-GP Large Capacity Hard Drives" (<http://products.wdc.com/library/whitepapers/eng/2579-771660.pdf>) (PDF). Western Digital. Retrieved 2013-04-20.
22. Smith, Roderick W. (2010-04-27). "Linux on 4KB-sector disks: Practical advice" (<http://www.ibm.com/developerworks/linux/library/l-4kb-sector-disks/>) . *DeveloperWorks*. IBM. Archived (<https://web.archive.org/web/20170824115221/https://www.ibm.com/developerworks/linux/library/l-4kb-sector-disks/>) from the original on 2017-08-24. Retrieved 2013-04-19.
23. "MBR (x86)" ([http://wiki.osdev.org/index.php?title=MBR_\(x86\)&oldid=12904](http://wiki.osdev.org/index.php?title=MBR_(x86)&oldid=12904)) . *OSDev Wiki*. OSDev.org. 2012-03-05. Archived ([https://web.archive.org/web/20170824003752/http://wiki.osdev.org/index.php?title=MBR_\(x86\)&oldid=12904](https://web.archive.org/web/20170824003752/http://wiki.osdev.org/index.php?title=MBR_(x86)&oldid=12904)) from the original on 2017-08-24. Retrieved 2013-04-20.
24. Sedory, Daniel B. (2003-07-30). "IBM DOS 2.00 Master Boot Record" (<http://daniel.sedory.com/asm/mbr/200MBR.htm>) . The Starman's Realm. Archived (<https://web.archive.org/web/20170824003921/http://thestarman.pcministry.com/asm/mbr/200MBR.htm>) from the original on 2017-08-24. Retrieved 2026-07-27.
25. Singh, Amit (2009-12-25) [December 2003]. "Booting Mac OS X" (http://osxbook.com/book/bonus/ancient/whatismacosx/arch_boot.html) . *Mac OS X Internals: The Book*. Retrieved 2011-07-22.

26. de Boyne Pollard, Jonathan (2011-07-10). "The EFI boot process" (<http://jdebp.eu/FGA/efi-boot-process.html>) . *Frequently Given Answers*. Archived (<https://web.archive.org/web/20170824004128/http://jdebp.eu/FGA/efi-boot-process.html>) from the original on 2017-08-24. Retrieved 2011-07-22.
27. Domsch, Matt (2005-03-22) [2003-12-19]. "Re: RFC 2.6.0 EDD enhancements" (<https://lkml.org/lkml/2003/12/19/139>) . *Linux Kernel Mailing List*. Archived (<https://web.archive.org/web/20170824004143/https://lkml.org/lkml/2003/12/19/139>) from the original on 2017-08-24. Retrieved 2017-08-24.
28. "Windows may use Signature() syntax in the BOOT.INI file" (<http://support.microsoft.com/kb/227704>) . *KnowledgeBase*. Microsoft.
29. McTavish (February 2014). "Vista's MBR Disk Signature" (<http://www.multibooters.co.uk/mbr.html>) . *Multibooters: Dual and Multibooting with Vista*. Archived (<https://web.archive.org/web/20170824120323/http://www.multibooters.co.uk/mbr.html>) from the original on 2017-08-24. Retrieved 2017-08-24.
30. Russinovich, Mark (2011-11-08). "Fixing Disk Signature Collisions" (<http://blogs.technet.com/b/markrussinovich/archive/2011/11/08/3463572.aspx>) . *Mark Russinovich's Blog*. Microsoft. Archived (<https://web.archive.org/web/20170824120420/https://blogs.technet.microsoft.com/markrussinovich/2011/11/06/fixing-disk-signature-collisions/>) from the original on 2017-08-24. Retrieved 2013-04-19.
31. Sakamoto, Masahiko (2010-05-13). "Why BIOS loads MBR into `0x7C00` in x86?" (<http://www.glamenv-septzen.net/en/view/6>) . *Glamenv-Septzen.net*. Archived (<https://web.archive.org/web/20170824002951/https://www.glamenv-septzen.net/en/view/6>) from the original on 2017-08-24. Retrieved 2011-05-04.
32. Compaq Computer Corporation; Phoenix Technologies Ltd.; Intel Corporation (1996-01-11). "BIOS Boot Specification 1.01" (<https://acpica.org/sites/acpica/files/specsbbs101.pdf>) (PDF). 1.01. ACPICA. Archived (<https://web.archive.org/web/20170824121219/https://acpica.org/sites/acpica/files/specsbbs101.pdf>) (PDF) from the original on 2017-08-24. Retrieved 2013-04-20. (<https://web.archive.org/web/20160414104624/http://www.phoenix.com/resources/specs-bbs101.pdf>)
33. Elliott, David F. (2009-10-12). "Why does the "standard" MBR set SI?" (<http://tgwbd.org/blog/2009/10/12/why-does-the-standard-mbr-set-SI/>) . *tgwbd.org*. Archived (<https://web.archive.org/web/20170824122006/http://tgwbd.org/blog/2009/10/12/why-does-the-standard-mbr-set-SI/>) from the original on 2017-08-24. Retrieved 2013-04-20.

34. Compaq Computer Corporation; Phoenix Technologies Ltd.; Intel Corporation (1994-05-05). "Plug and Play BIOS Specification 1.0A" (<https://web.archive.org/web/20170824121146/http://download.intel.com/support/motherboards/desktop/sb/pnpbiosspecificationv10a.pdf>) (PDF). 1.0A. Intel. Archived from the original (<http://download.intel.com/support/motherboards/desktop/sb/pnpbiosspecificationv10a.pdf>) (PDF) on 2017-08-24. Retrieved 2013-04-20.
35. Paul, Matthias R. (1997-10-02) [1997-09-29]. "Caldera OpenDOS 7.01/7.02 Update Alpha 3 IBMBIO.COM - README.TXT and BOOT.TXT - A short description of how OpenDOS is booted" (<https://web.archive.org/web/20031004074600/http://www-student.informatik.uni-bonn.de/~frinke/ibmbioa3.zip>) . Archived from the original (<http://www.uni-bonn.de/~uzs180/download/ibmbioa3.zip>) on 2003-10-04. Retrieved 2009-03-29. (<https://web.archive.org/web/20181225154705/http://mirror.macintosharchive.org/max1zzz.co.uk/+Windows%20&%20DOS/DOS/System/Novell/Support/Bins/Op702src.zip>)
36. Paul, Matthias R. (2017-08-14) [2017-08-07]. "The continuing saga of Windows 3.1 in enhanced mode on OmniBook 300" (<http://hpmuseum.org/forum/thread-8774-post-77196.html#pid77196>) . *MoHPC - the Museum of HP Calculators*. Archived (<https://web.archive.org/web/20180501185933/http://hpmuseum.org/forum/thread-8774-post-77196.html>) from the original on 2018-05-01. Retrieved 2018-05-01. "[...] SYS [...] /O[:nnn] Override IPL reported boot drive unit (n=0..126, 128..254). [...] Preparing target disk... Choosing FAT12 CHS Boot Sector (requires IPL to report boot unit). Treating target as diskette or superfloppy medium (boot drive unit 0). Writing new Boot Sector... [...]" (NB. SYS writes **volume boot records** rather than master boot records, but their incoming register interface is similar (with extensions) since they could both be loaded initially by the underlying system.)
37. Elliott, Robert (2010-01-04). "EDD-4 Hybrid MBR boot code annex" (http://www.t13.org/Documents/UploadedDocuments/docs2010/e09127r3-EDD-4_Hybrid_MBR_boot_code_annex.pdf) (PDF). Hewlett Packard, T13 Technical Committee. e09127r3. Archived (https://web.archive.org/web/20170824121640/http://www.t13.org/Documents/UploadedDocuments/docs2010/e09127r3-EDD-4_Hybrid_MBR_boot_code_annex.pdf) (PDF) from the original on 2017-08-24. Retrieved 2013-04-20.
38. "FDISK /MBR rewrites the Master Boot Record" (<http://support.microsoft.com/kb/69013>) . *Support*. 1. Microsoft. 2011-09-23. 69013. Archived (<https://web.archive.org/web/20170208102308/https://support.microsoft.com/en-us/help/69013/fdisk-mbr-rewrites-the-master-boot-record>) from the original on 2017-02-08. Retrieved 2013-04-19.
39. "sfdisk(8) – Linux man page" (<http://linux.die.net/man/8/sfdisk>) . *die.net*. 2013 [2007]. Archived (<https://web.archive.org/web/20170824121954/https://linux.die.net/man/8/sfdisk>) from the original on 2017-08-24. Retrieved 2013-04-20.

40. Brown, Ralf D. (2000-07-16). "Ralf Browns Interrupt List (v61 html)" (<http://www.delorie.com/djgpp/doc/rbinter/id/13/6.html>) . Delorie Software. Retrieved 2016-11-03.
41. Brown, Ralf D. (2000-07-16). "B-1302: INT 13 - DISK - READ SECTOR(S) INTO MEMORY" (<http://www.cs.cmu.edu/~ralf/interrupt-list/inter61b.zip>) . *Ralf Brown's Interrupt List (RBIL)* (61 ed.). Retrieved 2016-11-03. (NB. See file INTERRUPT.B inside archive "INTER61A.ZIP")

Дополнительная литература

- Гилберт, Говард (1 января 1996 г.) [1995]. "Перегородки и тома" (<https://web.archive.org/web/20160303210749/http://www.yale.edu/pclt/BOOT/PARTITIO.HTM>) . *PC Lube & Tune*. Архивировано из оригинала (<http://www.yale.edu/pclt/BOOT/PARTITIO.HTM>) 3 марта 2016 года.
- Найтс, Рэй (2004-12-22) [2000-12-16]. "Место Рэя" (https://web.archive.org/web/20170824133034/http://www.rayknights.org/pc_boot/pc_boot.htm) . *MBR и загрузочные сектора Windows (включая дизассемблированный код и пояснения к процессу загрузки)*. Архивировано из оригинала 24.08.2017. Проверено 2017-08-24.
- Лэндис, Хейл (2002-05-06). "Master Boot Record" (<https://web.archive.org/web/20140701052540/http://www.ata-atapi.com/hiwmbr.html>) . *Как это работает*. Архивировано из оригинала (<http://www.ata-atapi.com/hiwmbr.html>) 1 июля 2014 года.
- Седори, Дэниел Б. (2015-06-25) [2007]. "MBRs (Master Boot Records)" (<http://daniel.sedory.com/asm/mbr/index.html>) . *Boot Records Revealed*. Архивировано (<https://web.archive.org/web/20170824133321/http://thestarman.pcministry.com/asm/mbr/index.html>) из оригинала 24.08.2017. Дата обращения: 27 июля 2026. (<https://web.archive.org/web/20070510140812/http://mirror.href.com/thestarman/asm/mbr/index.html>) (<http://thestarman.narod.ru/asm/mbr/index.html>)

Внешние ссылки

- Статья о главной загрузочной записи ([http://wiki.osdev.org/MBR_\(x86\)](http://wiki.osdev.org/MBR_(x86)))
- MBR и ее роль в процессе загрузки BIOS (<https://neosmart.net/wiki/mbr-boot-process/>)